



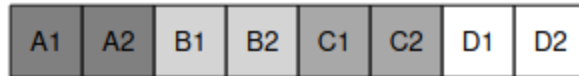
FAST FILE SYSTEM

Poor performance in the old UNIX File System

- **The Problem: Terrible File System Performance**
 - Only **2% of disk bandwidth** utilized (McKusick et al. [MJLF84])
 - Disk treated like **random-access memory** – no awareness of disk mechanics
- **Data scattered** across disk – costly seek operations
 - Inodes far from data blocks
- **Fragmented free space**
 - No effort to keep blocks contiguous
 - Leads to fragmented file allocation

Example

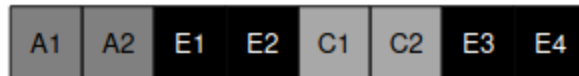
For example, imagine the following data block region, which contains four files (A, B, C, and D), each of size 2 blocks:



If B and D are deleted, the resulting layout is:



As you can see, the free space is fragmented into two chunks of two blocks, instead of one nice contiguous chunk of four. Let's say you now wish to allocate a file E, of size four blocks:



Effect on small block size

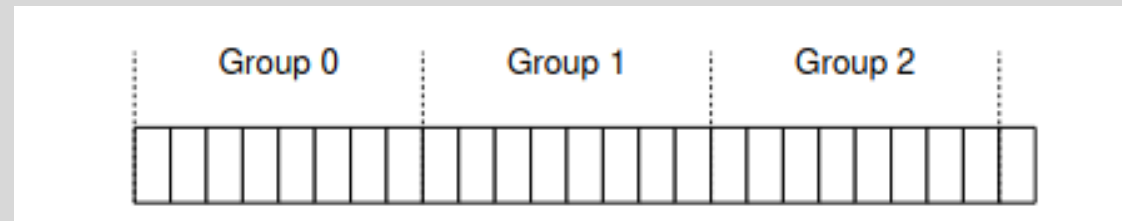
- **Small block size (512 bytes)**
 - Less internal waste, but high seek overhead
 - Inefficient data transfer

FFS: Disk Awareness Is the Solution

- Developed at **Berkeley** to fix poor UNIX file system performance
- Designed to be “**disk aware**”, optimized for real disk behavior
- Improved **block placement** and **data locality**
- Reduced **fragmentation** through smarter allocation
- Kept the same **file system interface** (e.g., `open()`, `read()`, `write()`)
- Enabled modern file systems to **evolve internally** without breaking apps

Organizing Structure: The Cylinder Group

- Disk Layout in FFS:
 - Disk divided into **cylinder groups**
 - Each **cylinder** = set of tracks on different surfaces at the same distance from the center
 - FFS aggregates **N consecutive cylinders** into one group
- Modern Drives:
 - Do **not expose geometry** (just logical block addresses)
 - File systems like **ext2/ext3/ext4** use **block groups** instead
 - Each block group = a **contiguous portion** of the disk address space
Example: 8 blocks per group



Locality and the Fast File System

- Within a Cylinder Group:
 - **Superblock (S)**: Stored in **each group** for redundancy
 - **Bitmaps**:
 - **Inode bitmap (ib)** and **Data bitmap (db)**
 - Efficient free space management
 - **Inode & Data Block Regions**:
 - Similar to those in VSFS
 - Most of the group = **data blocks**
- Why Use Groups?
 - Improves performance by maintaining **locality**
 - Storing files/directories in the same group avoids **long disk seeks**
 - Each group has all necessary metadata to manage itself



FFS Placement Policy: Keeping Related Stuff Together

- **Goal:** Improve performance by organizing files & metadata smartly.
 - **Mantra:**
 - **Keep related stuff together**
 - **Keep unrelated stuff apart**
- **Heuristics Used by FFS**
- **Directory Placement:**
 - Choose a **group with few directories** and **many free inodes**
 - Balances load & enables future file allocation
- **File Placement:**
 - Place file's **data blocks in the same group** as its inode
 - Place **all files in a directory** in the same group as the directory

Example: Good FFS Allocation

- Assume:
- 10 inodes & 10 data blocks per group
- 3 directories: /, /a, /b
- 4 files: /a/c, /a/d, /a/e, /b/f
- Files in same directory placed together
- Data near inodes
- Short seeks

group	inodes	data
0	/-----	/-----
1	acde-----	accddee---
2	bf-----	bff-----
3	-----	-----
4	-----	-----
5	-----	-----
6	-----	-----
7	-----	-----

Bad Allocation (Spreading Inodes)

- Files in /a spread across groups
- Longer seeks
- Poor locality

group	inodes	data
0	/-----	/-----
1	a-----	a-----
2	b-----	b-----
3	c-----	cc-----
4	d-----	dd-----
5	e-----	ee-----
6	f-----	ff-----
7	-----	-----

Measuring File Locality

- **Why Measure It?**

- To verify if FFS's **locality heuristics** make sense in practice: Do file accesses happen to **related files** in the **directory tree**?

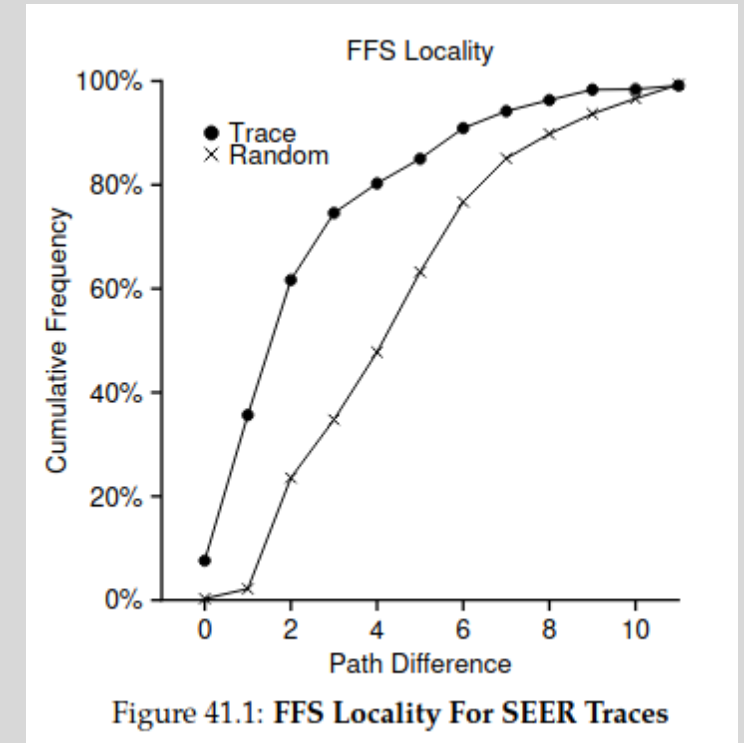
- **Locality Metric Definition**

- File accesses are analyzed using **SEER traces** [K94]
 - **Distance Metric:** Measures how far up the directory tree you must go to find the **common ancestor** between two consecutive file opens.

	Example	Access Pattern	Distance
	Same file	$f \rightarrow f$	0
◦	Same dir	$\text{dir}/f \rightarrow \text{dir}/g$	1
	Different dirs under same parent	$\text{proj}/\text{src}/\text{foo.c} \rightarrow \text{proj}/\text{obj}/\text{foo.o}$	2

Results: SEER Trace Locality

- ~7% of file accesses are to the **same file** (distance = 0)
- ~40% of file accesses are to **same file or same directory** (distance = 0 or 1)
- ~25% are to files **2 levels apart** (e.g., /src ↔ /obj under /proj)
-  Supports FFS's assumption: **Name-based locality is real**



File Locality

- **Limitations of FFS**

- FFS groups files **within the same directory**
- **Cross-directory locality** (e.g., `/src/foo.c` → `/obj/foo.o`) is **not optimized**
 - **Leads to more seeks** despite files being related by usage

- **Random Trace Comparison**

- Files accessed in **random order** from SEER data
- Shows **less locality**, as expected
- Still has some due to shared **root ancestor**

- FFS heuristics are backed by **real-world trace data**:

- Related files **are often accessed together**, especially within the same directory
- Justifies FFS's grouping strategy for **performance optimization**

The Large-File Exception in FFS

- Problem with Large Files

- Without special handling, a **large file fills one block group** completely
- This prevents placing **related files** in the same group
- Result: **Poor file-access locality** for other files

- FFS Solution for Large Files

- Allocate first N blocks (e.g., 12 direct blocks) in the **first block group**
- Subsequent chunks (e.g., indirect blocks) placed in **different block groups**
- Groups chosen to minimize utilization, **spreading the file across disk**

Example Without Exception

- Large file /a fills most of Group 0's data blocks
- Other groups mostly empty
- Limits space for new files in root directory

group	inodes	data
0	/a-----	/aaaaaaaaa aaaaaaaaaa aaaaaaaaaa a-----
1	-----	-----
2	-----	-----

Example With Large-File Exception (chunk size = 5 blocks)

- File spread over multiple groups
- Each group less heavily utilized
- More space left for other files

group	inodes	data
0	/a-----	/aaaaa-----
1	-----	aaaaa-----
2	-----	aaaaa-----
3	-----	aaaaa-----
4	-----	aaaaa-----
5	-----	aaaaa-----
6	-----	-----

Performance Trade-Off: Seek vs. Transfer Time

- Spreading file across groups increases **seek overhead**
- Can be offset by **amortization** — doing more data transfer per seek
- Example: Balancing Seek and Transfer Time, Spend half the time seeking and half the time transferring data, (this means 50% disk utilization from transfer speed)

Example: Spend half the time seeking and half the time transferring data

Given:

Seek time (positioning) = 10 milliseconds (ms)

Transfer speed = 40 megabytes per second (MB/s)

What chunk size is needed?

You want to transfer data for 10 ms to match 10 ms seek time.
Calculate how much data can be transferred in 10 ms at 40 MB/s:

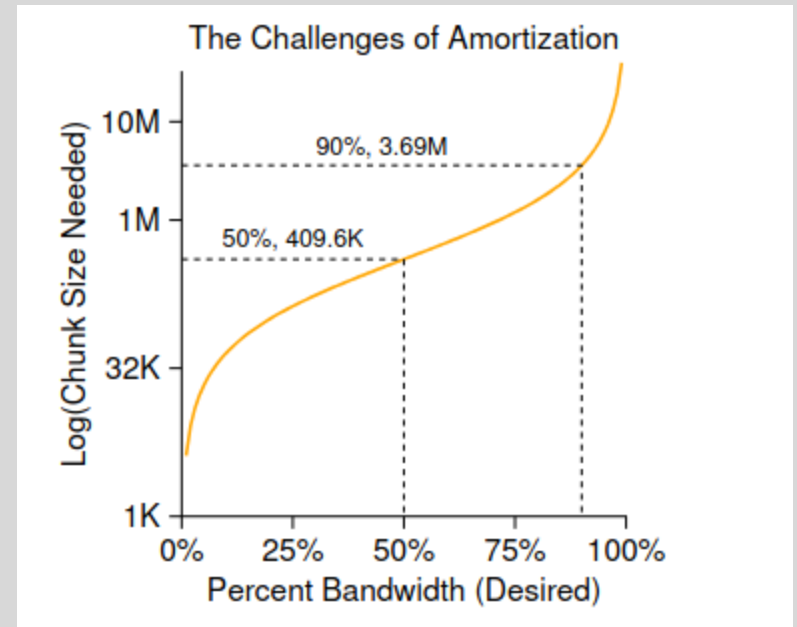
Data transferred = Transfer speed $\times$ Transfer time

$40 \text{ MB/s} \times 0.01 \text{ s} = 0.4 \text{ MB}$ or 409.6 KB

Interpretation

- To spend half the time seeking and half transferring, **each chunk should be about 409.6 KB**
- If chunks are smaller, seek overhead dominates → performance drops
- If chunks are larger, more time is spent transferring data → better throughput

Desired Transfer Time %	Chunk Size Needed
50%	409.6 KB
90%	~3.6 MB
99%	~39.6 MB



FFS Practical Approach

- FS does not use precise math like the chunk size calculation we just saw to decide how to split large files.
- Instead, it relies on the inode's internal structure to guide placement:
- The inode contains 12 direct block pointers
 - These point directly to data blocks.
 - These first 12 data blocks (each 4 KB) are stored in the same cylinder group as the inode itself.
 - So, the first 48 KB ($12 \text{ blocks} \times 4 \text{ KB/block}$) of the file stay close to the inode.
- After the direct blocks, the inode has indirect pointers:
 - The first indirect pointer points to a block that holds 1024 pointers to data blocks.
 - Each of these 1024 data blocks is 4 KB, so this indirect block covers 4 MB of data ($1024 \times 4 \text{ KB}$).
 - FFS places all the data blocks referenced by each indirect block into a different cylinder group from the inode and the previous blocks.

Hardware Trends Impact

- Disk **transfer rates improve quickly**
- Disk **mechanical seek times improve slowly**
- Means: Seek costs become more expensive relative to transfer
- Larger chunks needed to **amortize seek overhead effectively**

Small File Handling

- Many files were ~2KB; 4KB blocks caused internal fragmentation, wasting ~50% of disk space.
- FFS introduced **sub-blocks** of 512 bytes for small files.
- Small files occupy multiple sub-blocks instead of whole 4KB blocks.
- When file grows to 4KB, sub-blocks are combined into a full block.
- **Efficiency Concerns**
 - Copying sub-blocks can cause extra I/O overhead.
 - To avoid this, libc was modified to buffer writes and issue them in 4KB chunks.

Disk Layout Optimization

- Early disks required CPU control; sequential reads could cause full rotations waiting for next block.
- FFS staggered block placement by skipping blocks to allow time for reads, called **parameterization**.
- This prevented extra rotations by matching disk timing characteristics.

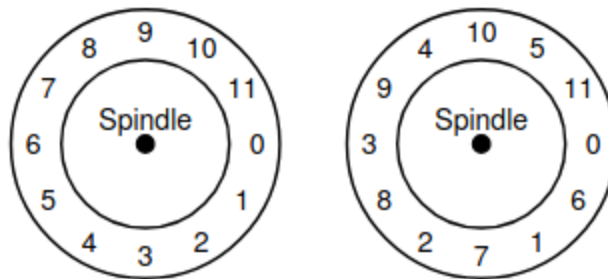
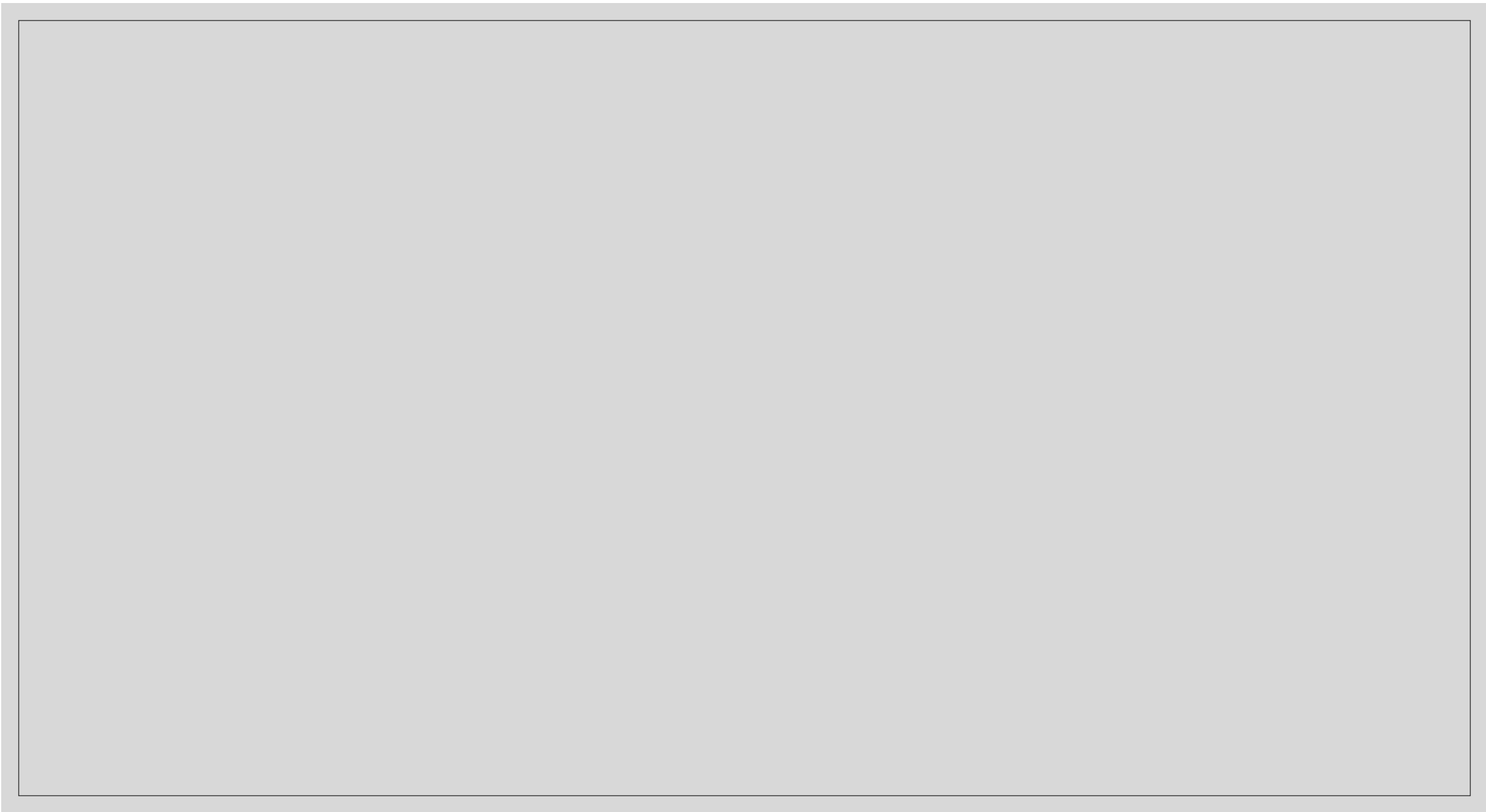


Figure 41.3: FFS: Standard Versus Parameterized Placement

Modern Disk Improvement and Layout Optimization

- Original FFS layout skipped blocks to avoid rotational latency
- Caused disks to read only **50% of peak bandwidth**
 - Disk made **two rotations** per track to read all blocks
- Modern disks use **internal track buffers**
 - Read entire track once and cache it internally
 - Subsequent reads served quickly from cache
- Result:
 - File systems no longer need to stagger block layout
 - Achieve much higher effective throughput and performance





Thank You